

5.3~ 5장 Part2

전이 학습

- 합성곱 신경망 기반의 딥러닝 모델 훈련을 위해서는 많은 양의 데이터가 필요

→ 큰 데이터셋 확보의 어려움을 해결한 것이 **전이 학습**

전이 학습이란, 이미지넷처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

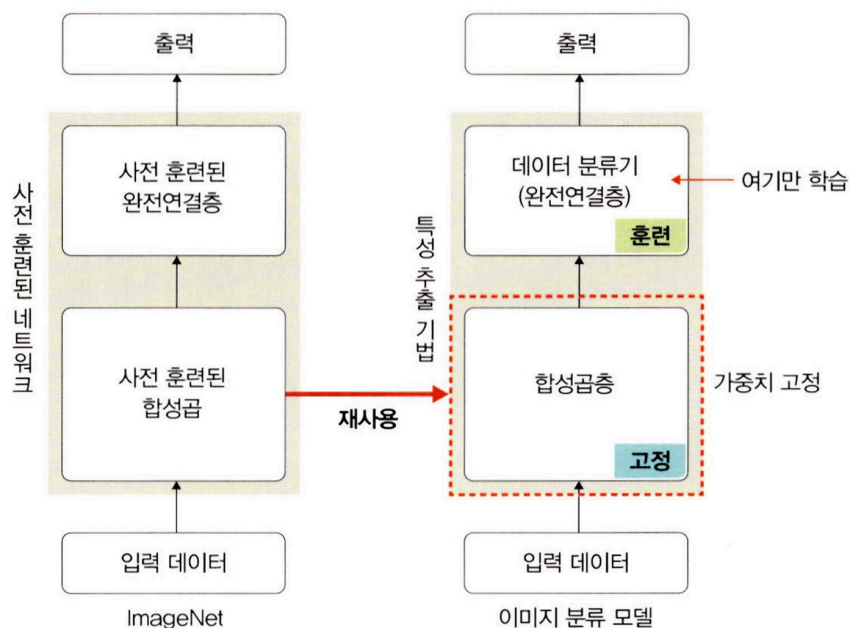
- 사전 훈련된 모델(네트워크): 아주 큰 데이터셋을 사용해 훈련된 모델
- 전이 학습을 위한 방법: 특성 추출 기법, 미세 조정 기법

특성 추출 기법

특성 추출(feature extractor)은 ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막 완전연결층 부분만 새로 만든다.

- 합성곱층: 합성곱층 + 풀링층
- 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스 분류

▼ 그림 5-30 특성 추출 기법



```
# 이미지 전처리 코드
data_path = "/content/drive/MyDrive/Colab Notebooks/catanddog/train"
transform = transforms.Compose(
    [
        transforms.Resize((225, 226)),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ]
)
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)
print(len(train_dataset))
```

이미지 전처리 코드 설명

`torchvision.transform` 은 모델의 입력으로 사용할 수 있게 이미지 데이터를 변환해줌.

```
transform = transforms.Compose([transforms.Resize([256, 256]),
                                transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
                                transforms.ToTensor()])
```

(a) 이미지 크기 조정
 (b) 이미지를 랜덤한 크기로 자름
 (c) 이미지를 랜덤하게 수평 뒤집기
 (d) 이미지 데이터를 텐서로 변환

`datasets.ImageFolder` 는 데이터로더가 데이터를 불러올 대상과 방법을 정의

```
train_dataset = torchvision.datasets.ImageFolder(data_path,
                                                    ① 데이터 경로
                                                    transform=transform)
                                                    ② 데이터의 전처리
```

데이터로더는 데이터를 불러오는 부분으로, 앞에서 정의한 ImageFolder을 데이터로더에 할당한다.

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32,
                                                    ① 데이터셋 지정          ②
                                                    num_workers=8, shuffle=True)
                                                    ③          ④ 데이터 셔플
                                                    사용할 프로세스의 수
```

```
# 학습에 사용될 이미지 출력
import numpy as np
import matplotlib.pyplot as plt
samples, labels = next(iter(train_loader))
classes = {0: 'cat', 1: 'dog'}
fig = plt.figure(figsize=(16,24))
for i in range(24): # 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

이미지 출력 코드 설명

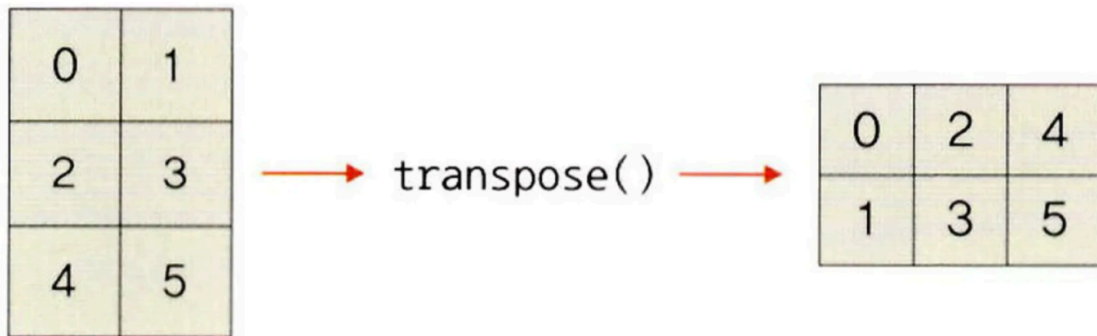
반복자 구현을 위해서는 `iter()` 와 `next()` 가 필요함

- `iter()` : 전달된 데이터의 반복자를 꺼내 반환
- `next()` : 그 반복자가 다음에 출력해야 할 요소를 반환

→ `train_loader`에서 데이터를 하나씩 꺼내오는 코드

`np.transpose` 는 행과 열을 바꿔 행렬의 차원을 바꿔줌

▼ 그림 5-33 transpose()



▼ 행렬의 차원 변환

행렬의 차원을 바꾸는 이유 → 행렬의 내적 연산 때문

▼ 그림 5-34 행렬의 내적

곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

- A 행렬의 행과 B 행렬의 열의 수가 같을 때만 내적이 가능함
- 그 둘이 다른데 내적을 하고 싶다면 np.transpose()나 np.reshape() 등으로 차원을 조정한다

```
# 변환 예시
exam = np.arange(24).reshape(2,3,4)
```



```
→ array([[[ 0, 1, 2, 3],
           [ 4, 5, 6, 7],
           [ 8, 9, 10, 11]],

          [[12, 13, 14, 15],
           [16, 17, 18, 19],
           [20, 21, 22, 23]]])
```

np.transpose(exam, (2, 1, 0)) # (2,3,4) 크기를 (4,3,2)로 변환

```
-> array([[[ 0, 12],
           [ 4, 16],
           [ 8, 20]],

          [[ 1, 13],
           [ 5, 17],
           [ 9, 21]],

          [[ 2, 14],
           [ 6, 18],
           [10, 22]],

          [[ 3, 15],
           [ 7, 19],
           [11, 23]]])
```

사전 훈련된 모델 내려받기

resnet18 = models.resnet18(pretrained=True) # pretrained=True는 사전 학습된 가중치를 사용하겠다는 뜻

```
⚡ /usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.1
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for '
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 113MB/s]
```

ResNet18



50개의 계층으로 구성된 합성곱 신경망

ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용해 훈련된 신경망으로
전이 학습에 사용되도록 사전 훈련된 모델을 제공함

```
# 사전 훈련된 모델의 파라미터 학습 유무 지정
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False
set_parameter_requires_grad(resnet18)
```

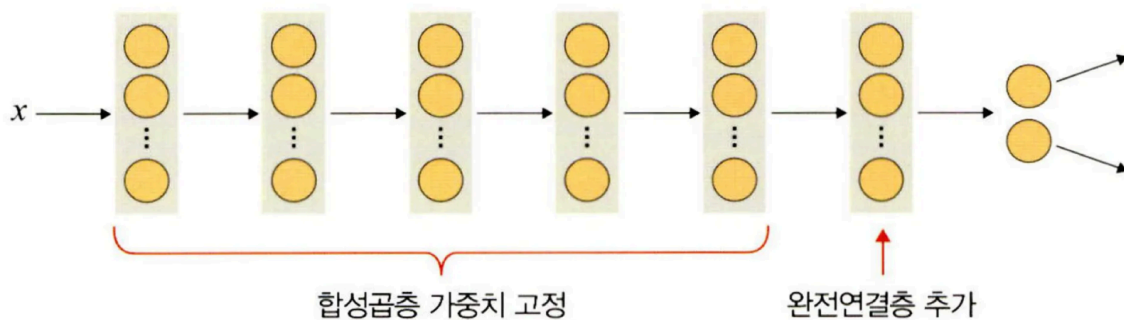
```
param.requires_grad = False
```

: 역전파 중 파라미터들에 대한 변화를 계산할 필요가 없음을 나타냄.

모델의 일부(합성곱층, 풀링층)를 고정하고 나머지를 학습하고자 할 때 설정함

```
# ResNet18에 완전 연결층 추가
resnet18.fc = nn.Linear(512, 2) # 2: 클래스가 2개
```

▼ 그림 5-36 ResNet18에 완전연결층 추가



```
# 모델의 파라미터 값 확인
```

```
for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)
```

```
fc.weight tensor([[ 0.0314, -0.0299, -0.0230, ..., 0.0064, 0.0007, -0.0355],
                  [ 0.0401, -0.0415, 0.0286, ..., 0.0041, -0.0051, 0.0246]])
fc.bias tensor([ 0.0149, -0.0367])
```

```

▶ # 모델 객체 생성 및 손실 함수 정의
model = models.resnet18(pretrained=True) # 모델의 객체 생성

for param in model.parameters(): # 모델의 합성곱층 가중치 고정
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters(): # 완전연결층은 학습
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실 함수 정의
print(model)

```

```

def train_model(model, dataloaders, criterion, optimizer, device, num_epoc
hs=13, is_train=True):
    since = time.time()
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # 데이터로더에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad() # 기울기를 0으로 설정
            outputs = model(inputs) # 순전파 학습
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward() # 역전파 학습
            optimizer.step()

            running_loss += loss.item() * inputs.size(0) # 출력 결과와 레이블 오차

```

를 계산한 결과 누적 저장

```
running_corrects += torch.sum(preds == labels.data) # 출력 결과와 레  
이블 동일한지 검사
```

```
epoch_loss = running_loss / len(dataloaders.dataset) # 평균 오차  
epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 평  
균 정확도
```

```
print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))
```

```
if epoch_acc > best_acc:  
    best_acc = epoch_acc
```

```
acc_history.append(epoch_acc.item())  
loss_history.append(epoch_loss)  
torch.save(model.state_dict(), os.path.join('/content/drive/MyDrive/Col  
ab Notebooks/catanddog', '{0:0=2d}.pth'.format(epoch)))  
print()
```

```
time_elapsed = time.time() - since  
print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time  
_elapsed % 60))  
print('Best Acc: {:.4f}'.format(best_acc))  
return acc_history, loss_history
```

```
# 파라미터 학습 결과를 옵티마이저에 전달  
params_to_update = []  
for name, param in resnet18.named_parameters():  
    if param.requires_grad==True:  
        params_to_update.append(param) # 파라미터 학습 결과 저장  
        print("###", name)  
optimizer = torch.optim.Adam(params_to_update) # 학습 결과를 옵티마이저에 전달
```



```
fc.weight  
fc.bias
```



모델 학습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
criterion = nn.CrossEntropyLoss() # 손실 함수 지정  
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion,  
                                                optimizer, device)
```

→ 약 93%로 높은 정확도를 보여줌

테스트 데이터로 정확도 검사

```
# 테스트 데이터 호출 및 전처리
test_path = "/content/drive/MyDrive/Colab Notebooks/catanddog/test"

transform = transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor()
    ]
)
test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32,
    num_workers=1,
    shuffle=True
)

print(len(test_dataset))
```

↺ 98

테스트 데이터 평가 함수 생성

```
def eval_model(model, dataloaders, device):
```

```
    since = time.time()
```

```
    acc_history=[]
```

```
    best_acc = 0.0
```

```
    saved_models = glob.glob("/content/drive/MyDrive/Colab Notebooks/cat
anddog" + "/*.pth")
```

```
    saved_models.sort() # 불러온 .pth 파일들 정렬
```

```
    print('saved_model', saved_models)
```

```
    for model_path in saved_models:
```

```
        print('Loading model', model_path)
```

```
        model.load_state_dict(torch.load(model_path))
```

```
        model.eval()
```

```
        model.to(device)
```

```

running_corrects = 0

for inputs, labels in dataloaders:
    inputs = inputs.to(device)
    labels = labels.to(device)

    with torch.no_grad(): # autograd 사용X
        outputs = model(inputs) # 모델에 데이터 적용 결과를 outputs에 저장
        _, preds = torch.max(outputs.data, 1)
        preds[preds >= 0.5] = 1 # torch.max로 출력된 값이 0.5 보다 크면 -> 올바른
예측
        preds[preds < 0.5] = 0 # torch.max로 출력된 값이 0.5 보다 작으면 -> 틀린
예측
        running_corrects += preds.eq(labels.cpu()).int().sum()
    epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 테스트
데이터의 정확도 계산
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc
        acc_history.append(epoch_acc.item())
    print()

time_elapsed = time.time() - since
print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_
elapsed % 60))
print('Best Acc: {:.4f}'.format(best_acc))

return acc_history # 계산된 정확도 반환

```

`glob` 은 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용함

`torch.max` 는 주어진 텐서 배열의 최댓값이 들어 있는 index를 반환

→ test 데이터도 94% 정도의 높은 정확도

예측 이미지 출력

```

# 예측 이미지 출력을 위한 전처리 함수
def im_convert(tensor):

```

```
image = tensor.clone().detach().numpy()
image = image.transpose(1, 2, 0)
image = image * np.array((0.5, 0.5, 0.5)) + np.array((0.5, 0.5, 0.5))
image = image.clip(0, 1)
return image
```

`tensor.clone()` : 기존 텐서의 내용을 복사한 텐서를 생성

`detach()` : 기존 텐서에서 기울기가 전파되지 않는 텐서

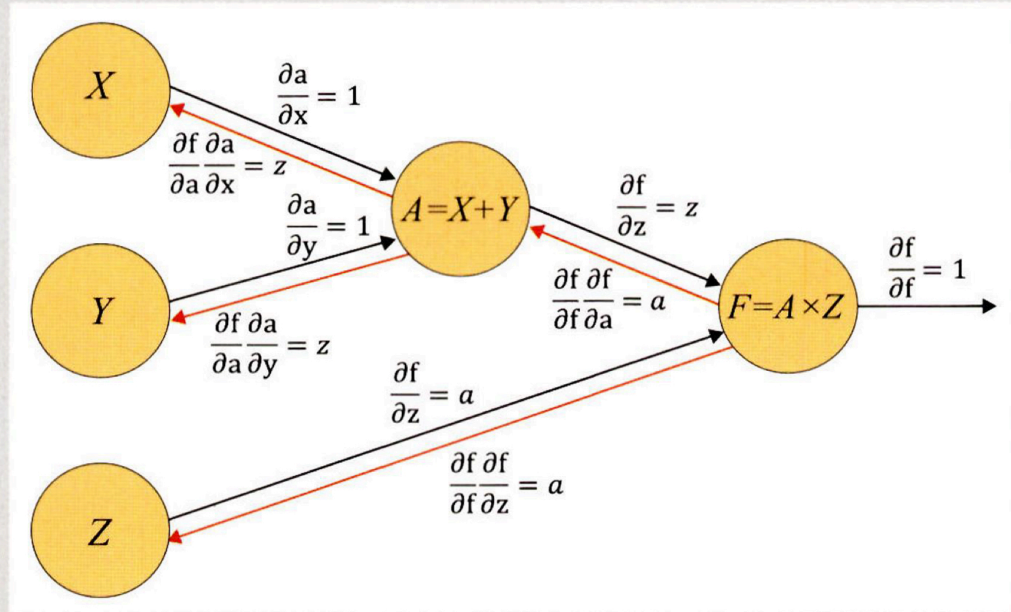
→ `tensor.clone().detach()` : 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에는 영향X

`clip()` : 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한함.

계산 그래프를 사용하는 이유

- 계산 그래프: 계산 과정을 그래프로 나타낸 것. 여러 개의 노드와 그 노드를 연결하는 선인 엣지로 구성됨

▼ 그림 5-39 계산 그래프



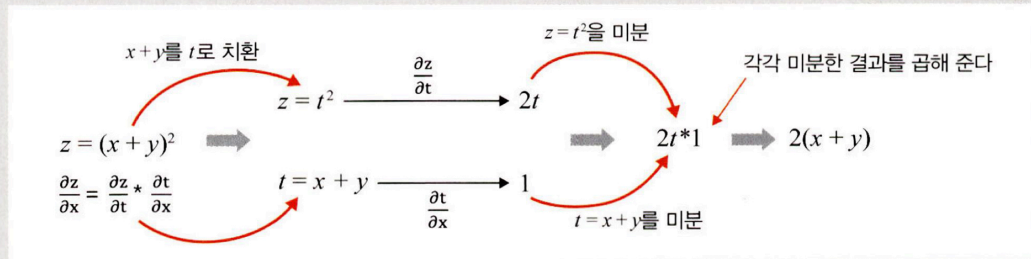
- 계산 그래프를 사용하는 두 가지 이유
 - 국소적 계산이 가능하다. 그림에서 Z 값이 바뀐다면, X, Y 계산 결과는 유지한 채로 바뀐 Z 연산이 필요한 $F = A \times Z$ 만 계산하면 된다
 - 역전파를 통한 미분 계산이 편리하다. 연쇄 법칙을 이용하여 빠르고 간편하게 미분을 계산할 수 있다.

연쇄 법칙

두 개 이상의 함수가 결합된 함수, 즉 합성 함수의 미분법을 연쇄 법칙 혹은 체인 룰(chain rule)이라고 한다.



▼ 그림 5-40 합성 함수의 미분



개와 고양이 예측 결과 출력

classes = {0: 'cat', 1: 'dog'} # 개와 고양이 두 개에 대한 레이블

dataiter = iter(test_loader) # 테스트 데이터셋을 가져옴

images, labels = dataiter.next() # 테스트 데이터셋에서 이미지와 레이블을 분리해 가져옴

output = model(images)

_, preds = torch.max(output, 1)

fig = plt.figure(figsize=(25, 4))

for idx in np.arange(20):

ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])

plt.imshow(im_convert(images[idx])) # 이미지 출력을 위해 앞서 정의한 im_convert 함수 적용

ax.set_title(classes[labels[i].item()])

ax.set_title("{}{}".format(str(classes[preds[idx].item()]), str(class[labels[idx].item()])))

item()])), color=("green" if preds[idx]==labels[idx] else "red"))

plt.show()

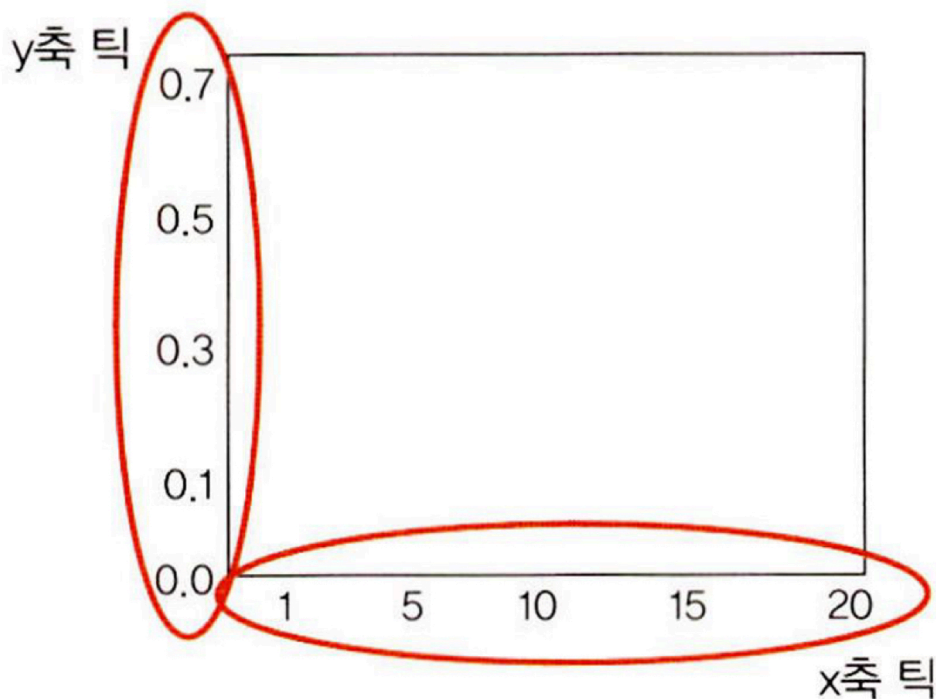
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)

add_subplot : 한 화면에 여러 개의 이미지를 담기 위해 사용함

```
add_subplot(2, 10, idx+1, xticks=[], yticks=[])
```

(a) (b) (c) (d) **틱을 삭제**
 이미지 크기 행 수 / 열 수 인덱스

▼ 그림 5-41 x축, y축 틱



`classes[preds[idx].item()]` : `preds[idx].item()` 값의 클래스를 판별하여 출력함

`subplots_adjust` : Figure 안에서 서브플롯의 위치를 조정

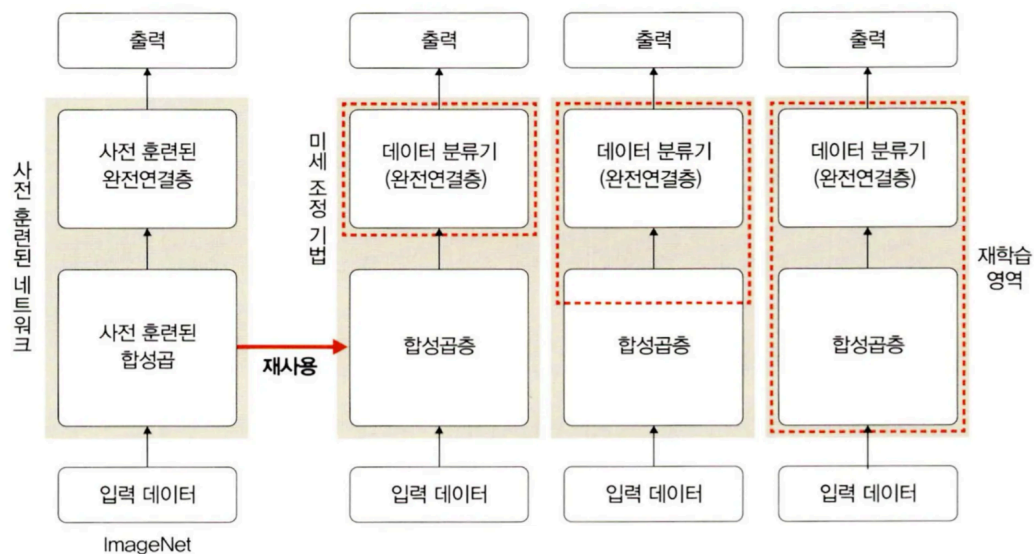
미세 조정 기법

- 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련 시키는 방식
- 특성이 잘못 추출되었다면 미세 조정 기법으로 새로운 이미지 데이터를 사용하여 네트워크의 가중치를 업데이트해서 특성을 다시 추출한다
- 미세 조정 기법은 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정
- 많은 연산량 필요 → GPU 사용 권장

미세 조정 기법 전략

- 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우 : 모델 전체를 재학습
- 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우: 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기를 학습
- 데이터셋 작고 유사성 작을 경우: 합성곱층 일부분과 데이터 분류기 학습
- 데이터셋 작고 유사성 클 경우: 데이터 분류기만 학습

▼ 그림 5-44 미세 조정 기법



설명 가능한 CNN

CNN은 입출력 사이의 과정이 마치 블랙박스과 같아 내부에서 어떻게 동작하는지 알기 어렵다.

→ 그 과정을 알기 위해서 CNN 처리 과정을 **시각화**

CNN의 시각화 방법 : 필터에 대한 시각화, 특성 맵에 대한 시각화

특성 맵 시각화

- 특성 맵(feature map): 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
- 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있게 도움
- PIL: 강력한 이미지 처리와 그래픽 기능을 제공하는 이미지 분석 및 처리 라이브러리

1. 네트워크 생성

: 13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크를 생성

- 코드가 길기 때문에 코랩 노트북 코드를 참고한다.

`log_softmax()` : 신경단 말단의 결괏값들을 확률 개념으로 해석하기 위해 소프트맥스 함수 결과에 log 값을 취한 연산 → 기울기 소멸 문제 해결을 위해 사용한다.

2. 모델 객체화

```
# 모델 객체화
model = XAI() # 객체 생성
model.to(device) # 장치에 모델 할당
model.eval() # 평가용으로 사용
```

3. 특성맵 결과 확인 함수 정의

- 특성 맵: 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과
- 합성곱층에서 입출력을 알 수 있다면 특성 맵에 대한 값들 확인 가능함.

```
# 특성 맵을 확인하기 위한 클래스 정의
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self): # hook 삭제
        self.hook.remove()
```

`hook` : 파이토치는 매 계층마다 hook 기능을 사용해 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있다.

`register_forward_hook` : 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져옴

4. 특성 맵 정보를 가져올 이미지 호출

```
# 이미지 호출
img=cv2.imread("/content/drive/MyDrive/Colab Notebooks/catanddog/trai
```

```

n/Cat/0.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
print(img.shape)

```

`cv2.resize` : 이미지 크기를 변경할 때 사용

- 보간법: 이미지상에 존재하는 픽셀 데이터들에 대해 근사함수를 적용해 새로운 픽셀 값을 구하는 것 → 추정해야 하는 픽셀을 보간법을 이용해 값 할당

```

cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
           (a)      (b)                        (c)

```

① 첫 번째 파라미터: 변경할 이미지 파일

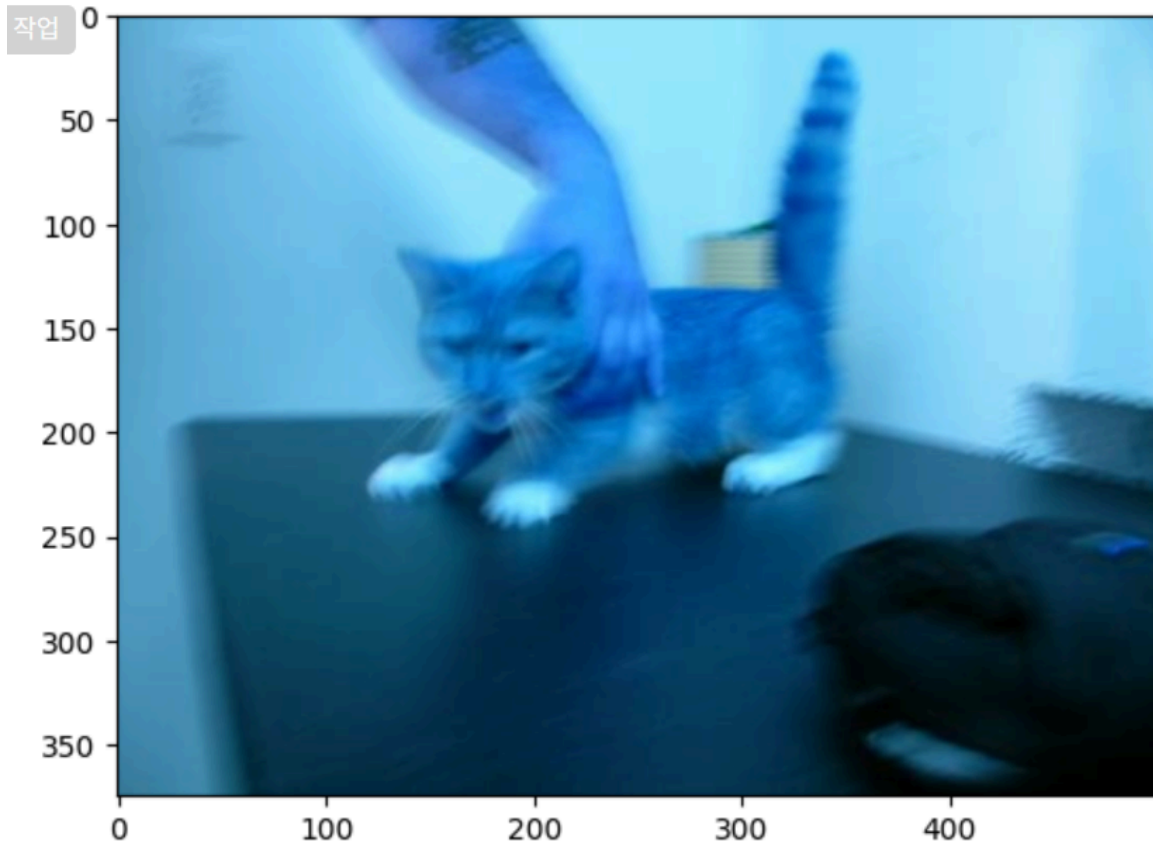
② 두 번째 파라미터: 변경될 이미지 크기를 (너비, 높이)로 지정

③ interpolation: 보간법

`unsqueeze()` : 1차원 데이터를 생성하는 함수

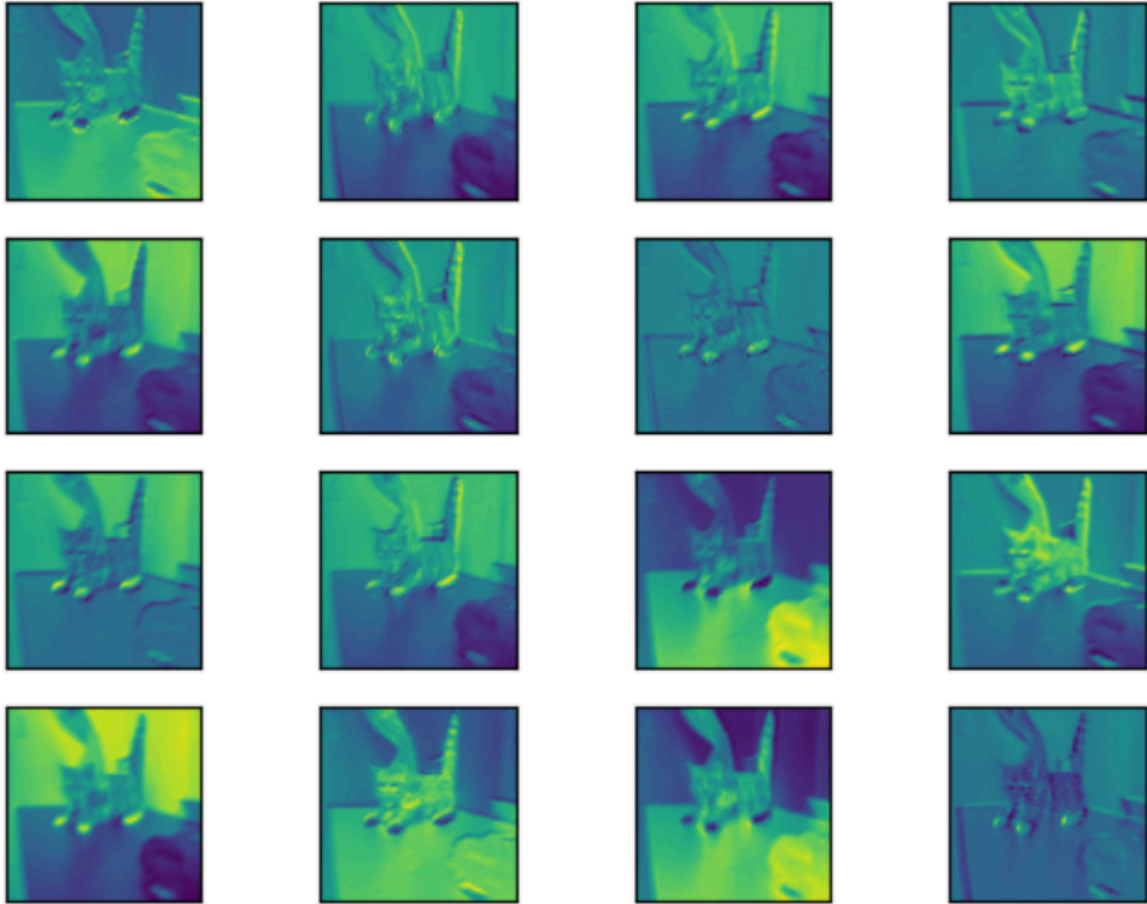
`ToTensor()(img).unsqueeze(0)` : 이미지를 텐서로 변환하고, 변환된 데이터를 1차원으로 변환함

```
torch.Size([1, 3, 100, 100])
```



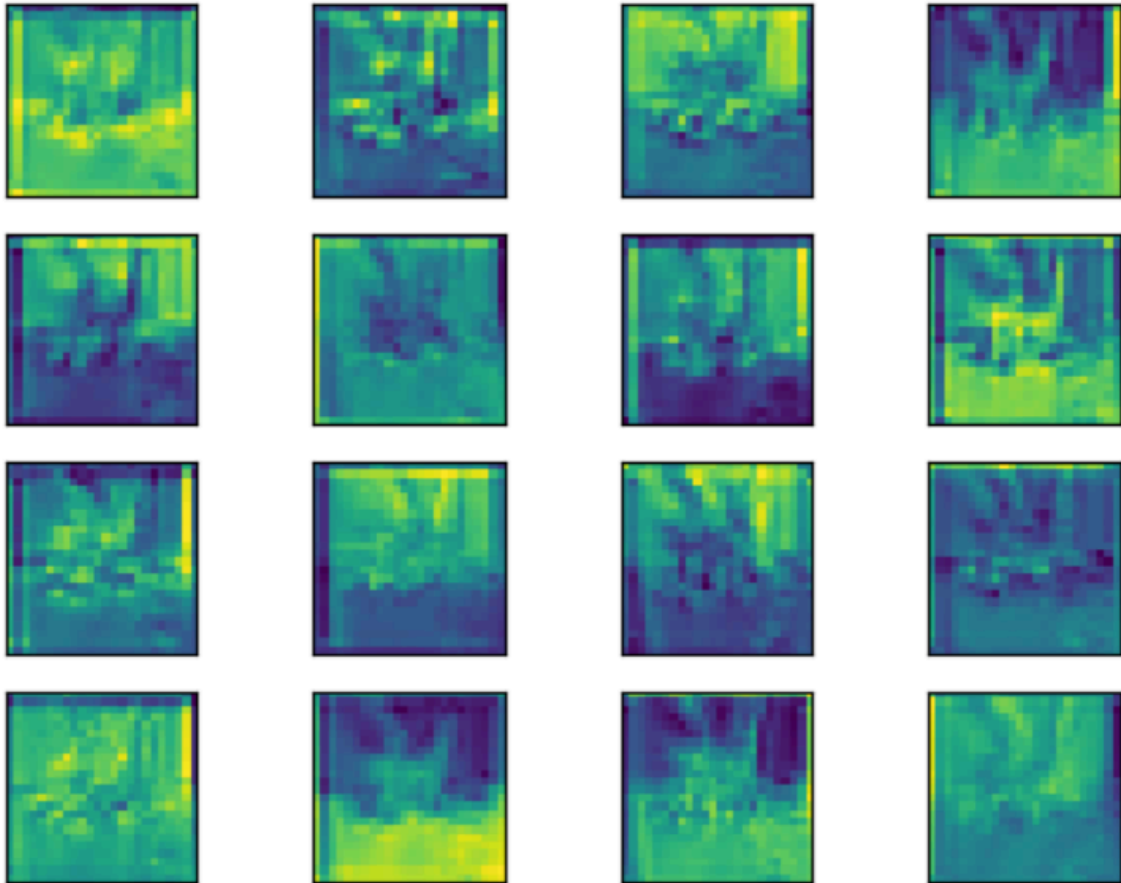
5. 특성 맵 확인

5.1. 첫 번째 Conv2d 계층에서 특성 맵



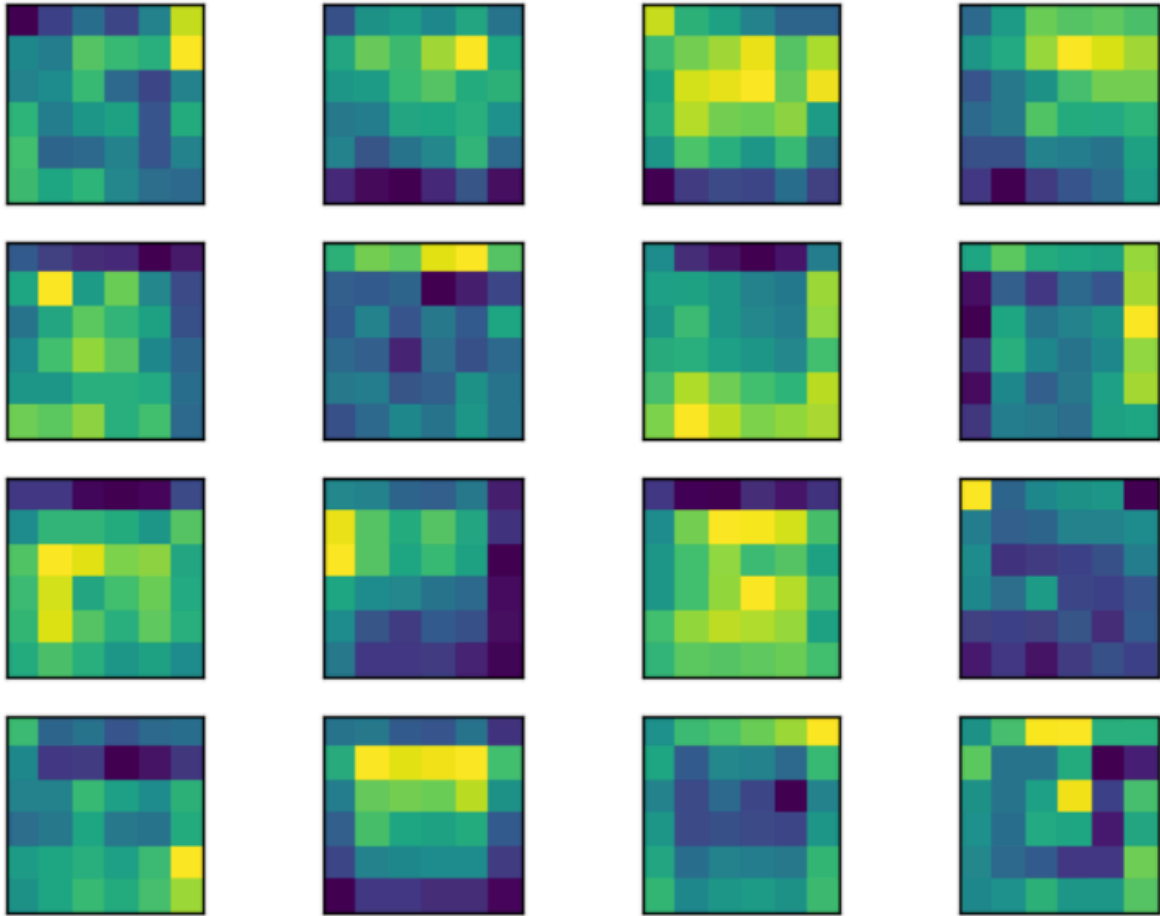
<Figure size 1200x800 with 0 Axes>

5.2 20번째 계층에 대한 특성 맵



<Figure size 1200x800 with 0 Axes>

5.3 40번째 계층에 대한 특성 맵



<Figure size 1200x800 with 0 Axes>

⇒ 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징들만 전달되는 것을 알 수 있다.

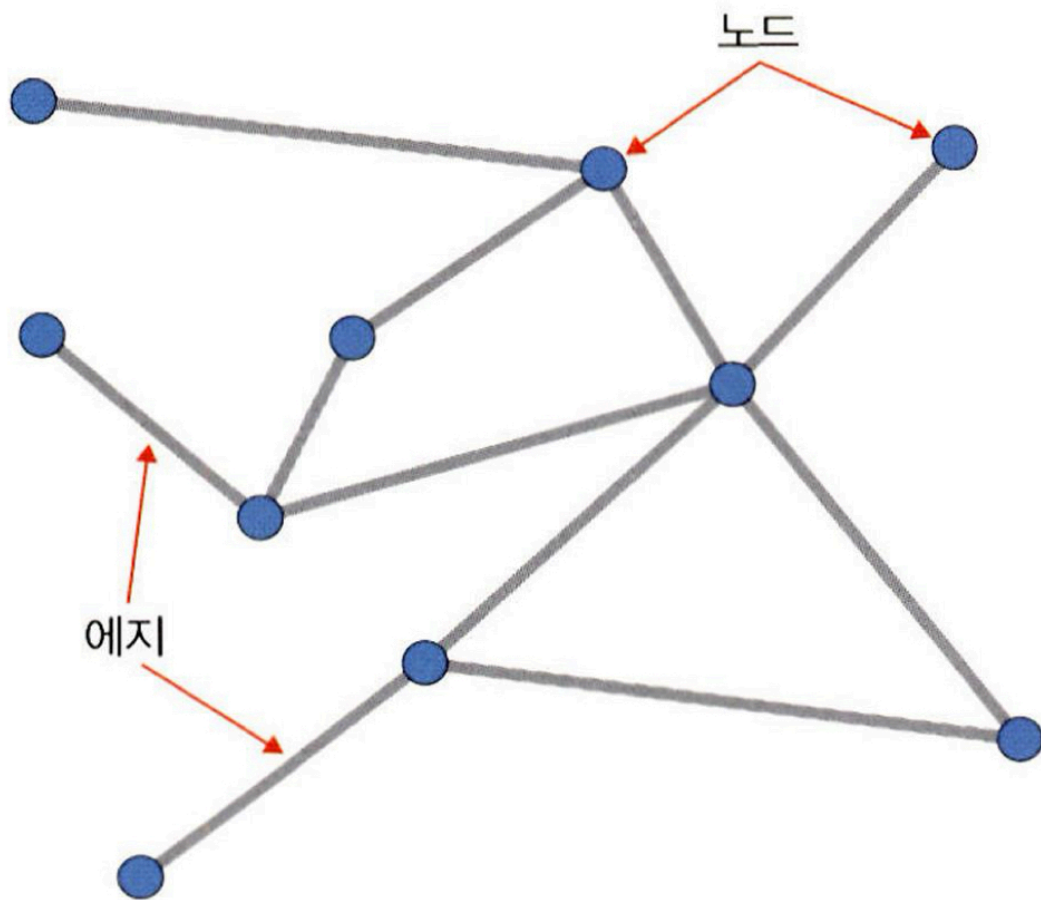
그래프 합성곱 네트워크

| 그래프 데이터를 위한 신경망

그래프

| 방향성이 있거나 없는 엣지로 연결된 노드의 집합

▼ 그림 5-50 그래프



- 그래프의 구성 요소: 노드, 엣지
- 노드: 원소
- 엣지: 결합 방법 (ex. single, double, triple, aromatic...)

그래프 신경망

| 그래프 구조에서 사용하는 신경망

그래프 데이터에 대한 표현 단계

1단계. 인접 행렬(adjacency matrix)

컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정

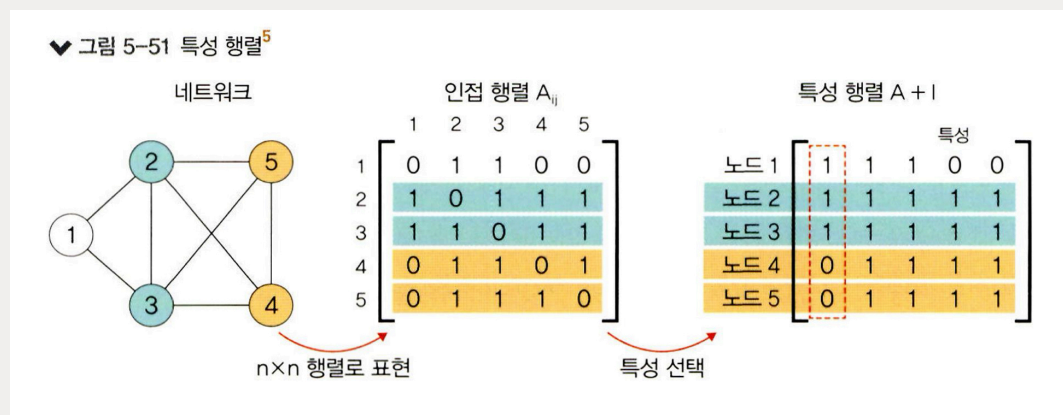
- 노드 n 개를 $n \times n$ 행렬로 표현
- 이렇게 생성된 인접 행렬 내의 값은 ' A_{ij} 는 i 와 j 의 관련성 여부'를 만족하는 값으로 채워줌

2단계. 특성 행렬(feature matrix)



- 인접 행렬만으로 특성 파악이 어려워 단위 행렬을 적용
- 각 입력 데이터에서 이용할 특성을 선택
- 특성 행렬에서의 각행: 선택된 특성에 대해 각 노드가 갖는 값 (ex. 첫 번째 행은 첫 번째 노드의 특성 값)

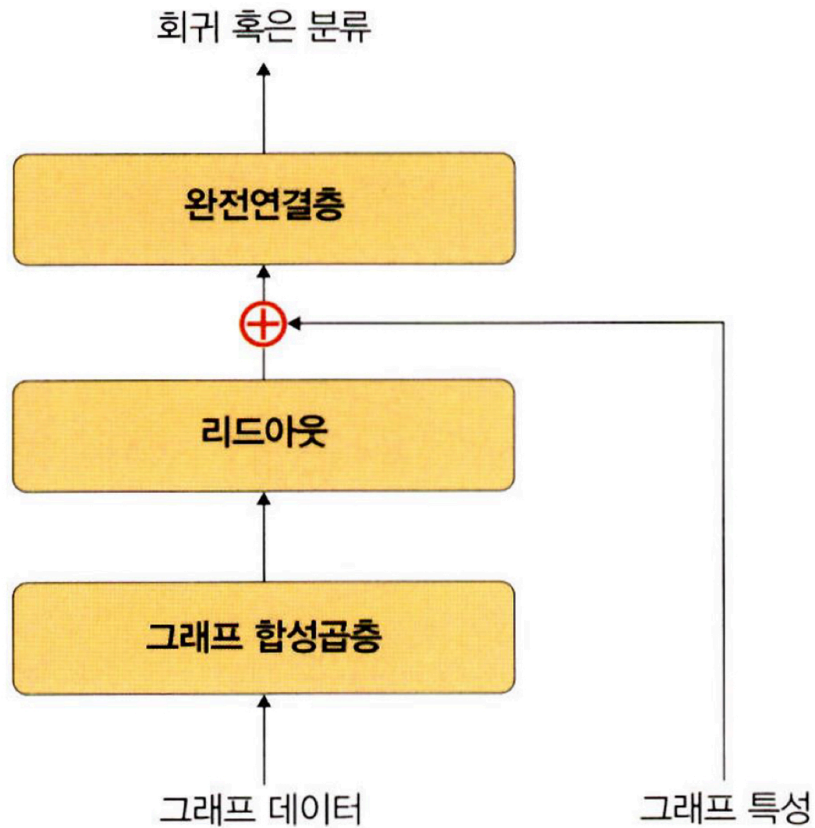
→ 특성 행렬 과정을 거쳐 그래프 특성이 추출됨



그래프 합성곱 네트워크 (GCN)

이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

▼ 그림 5-52 그래프 합성곱 네트워크



- 리드아웃: 특성 행렬을 하나의 벡터로 변환하는 함수

→ 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성

- 그래프 합성곱층: GCN에서 가장 중요한 부분, 이 부분을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있음

GCN의 활용 분야

- SNS에서 관계 네트워크
- 학술 연구에서 인용 네트워크
- 3D Mesh